A WORKING SESSION · 90 MINUTES

Coding with agents

You will not be writing code. You will be writing *instructions*, and learning where they go wrong.

Bring the laptop. Everything here is done, not watched.

Open the folder for this session

1. **File → Open Folder**
2. choose **1-monday-agents** — this folder, not the whole repository
3. open the **agent panel**
4. nothing to configure — it **already asks** before it edits a file or runs a command

the folder `atrium-school-ml-lessons / 1-monday-agents`

Everything the agent can read — the instructions, the skills, the data — is **scoped to the folder you open**. That is the first true thing about agents, and you have just used it.

Every turn re-sends the whole conversation

What does this Python script do?

It reads a CSV of finds and plots them by year.

Can it group them by site as well?

Yes — here is a version that groups by site…

Now add the map.

Here is the script with the map added…

Why is the map empty?

The coordinates are in a different projection…



Until it does not fit.

This single fact explains almost every strange thing an agent will do to you today.

Three consequences

It gets slower and dearer

every turn re-reads everything said so far

It starts losing the thread

the beginning is furthest from the question

And then it simply stops

full is full

None of that is you using it wrong.
It is the shape of the thing.

BLOCK 1

The agent loop

One idea. It fits on a single diagram, and the diagram runs.

0 1 2 3 4 5 6 7 8

0:07 — 13 minutes

The loop: decide, act, read the result, repeat

you	the map comes out empty — fix it
model	I need to see the script first
tool call	`read_file("plot_finds.py")`
result	`118 lines`
model	the coordinates are in EPSG:5514
tool call	`read_file("finds.csv")`
result	`first 20 rows`
tool call	`edit_file("plot_finds.py")`
result	`1 change applied`
tool call	`run("python plot_finds.py")`
result	`wrote map.png`
model	the projection was wrong; fixed and re-run



It stops when there is nothing left to ask for.

Not when it has said enough — when the job is done or it is stuck.

Chat and agent, side by side

Chat

- one question, one answer
- then it **stops and waits for you**
- it produces **text**
- you do the work

`for understanding, drafting, explaining`

Agent

- one question, many steps
- it **stops when the job is done**
- it produces **changed files**
- you check the work

`for doing, checking, repeating`

Most complaints about AI are the wrong shape used for the job.

TRY IT NOW · 6 MINUTES

Watch the loop turn

1. In the agent panel, give it the sentence below. **Nothing else.**
2. **Do not read the answer.** Watch the tool calls appear as it works.
3. **Expand three of them.** What did it ask for, and what came back?
4. Count them. **How many turns did your task take?** Ask your neighbour.

+ Finished early? Ask it to run **scripts/plot_finds.py** and say why the map is empty.

```
> What is in this folder? Describe each file in one sentence.
```

BLOCK 2

Tools, permissions and MCP

What a tool call really is, what happens when you refuse one, and how to give it tools it did not come with.

0 1 2 3 4 5 6 7 8

0:20 — 17 minutes

What a tool call is

what the model writes

```
read_file
path:  "plot_finds.py"
lines: 1–120
```

— a request. Nothing has happened yet.

what Zed does with it

opens the file and reads it

on your machine, with your permissions,
in your folder

what goes back into the conversation

`118 lines of plot_finds.py …`

appended as an ordinary message — and it counts against the window

a 3 000-line file read “just to check” costs you the rest of the session

Which is why tool results are expensive.

Everything a tool returns lives in the conversation for the rest of the session.

The tools an agent has

Read a file all of it, or a range of lines	**Search** by name, or by what is inside	**List a folder** what is here?
Fetch a page documentation, mostly	**Edit a file** this one changes your work	**Run a command** this one can do anything

Four of these only **look**. Two of them **act**. Almost everything worth worrying about is in those two.

What happens when it wants to act

what you see **a tool card, in the panel**

Edit file `scripts/plot_finds.py`

wants to change 1 file

Allow once

Deny once

Always for this tool

Always for a pattern

- reading and searching **run by themselves**
- editing a file and running a command **stop and ask**
- that is the default — **there is nothing to turn on**
- **Allow once** is what you use all afternoon
- the two **Always** options are how you go faster, and how you lose the brake

if you ever want to change it

```
agent.tool_permissions
  allow    run it, never ask
  deny     refuse it, never ask
  confirm  ask me  ← the default
```

Denying is not an error. **It goes back into the conversation, and the agent tries another way.**

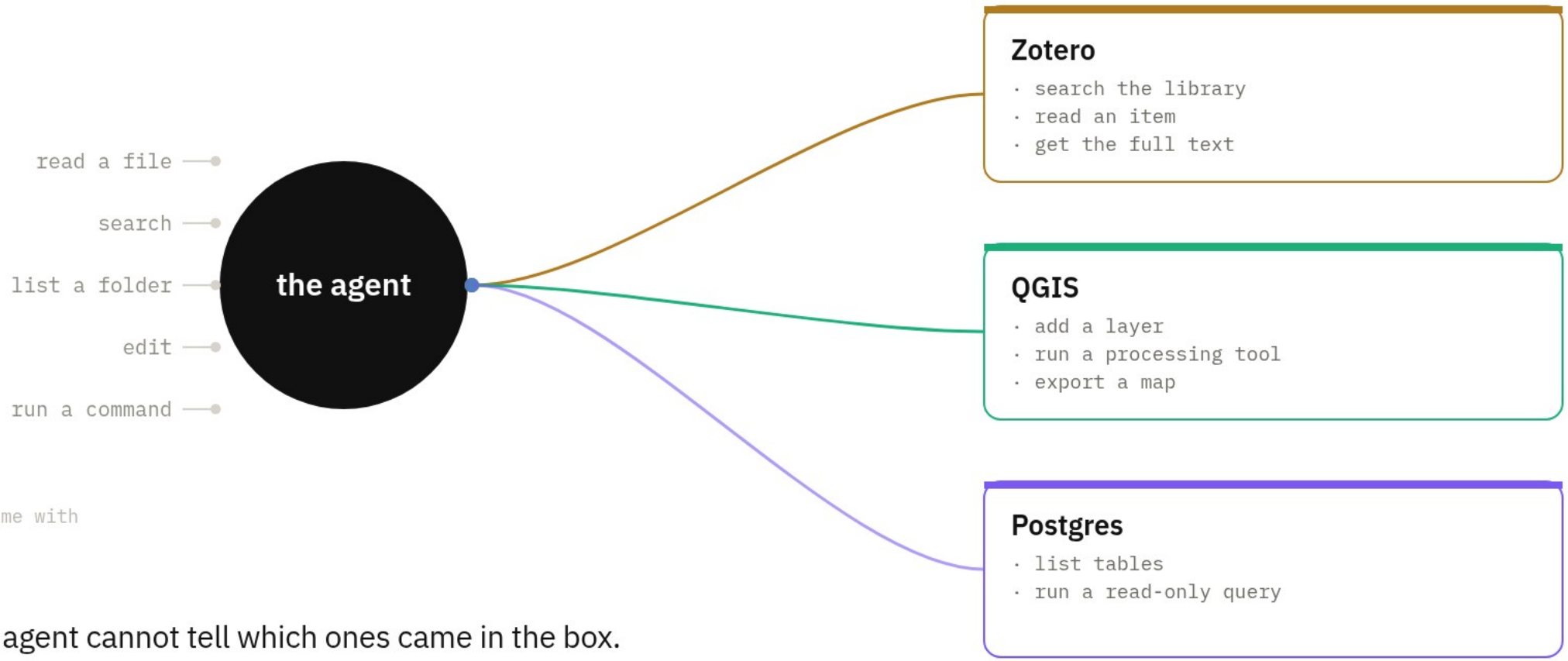
TRY IT NOW · 4 MINUTES

Say no, and watch what it does

1. Ask it to make the change below.
2. A tool card appears. Choose **Deny once**.
3. Read what it does next. **It does not give up — it changes approach.**
4. Now let it through with **Allow once**, and check what it changed.

```
> Rename the column “datum” to “date” everywhere in this folder.
```

MCP adds tools the agent did not come with



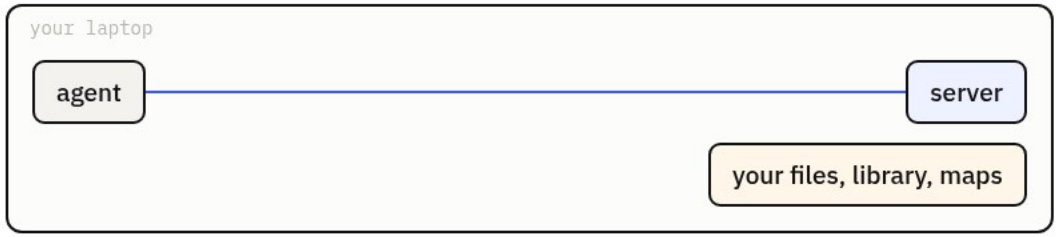
what it came with

And the agent cannot tell which ones came in the box.

It asks for “search the library” exactly the way it asks to read a file.

Local and remote servers

Local — you run it



- you install it and keep it updated
- it reaches **your own things**
- no account, no subscription
- nothing leaves the machine
- exists only on **this** laptop

`Zotero · QGIS · your files`

Remote — somebody else runs it



- nothing to install
- it reaches **their data**
- you sign in — sometimes you pay
- your questions go to them
- works from **any** laptop

`scite · hosted services`

Which one? **Ask where the data lives.** Your library and your maps are here. The world's literature is not.

Three MCP servers worth having

Zotero — your library

- search your library
- read an item's metadata
- pull the full text and your annotations

`local · community project`

QGIS — your maps

- add and remove layers
- run processing tools
- export a map

`local · QGIS plugin + server`

scite — the literature

- search full text
- how many citations **supported** a claim
- how many **contrasted** it

`remote · account needed`

Files

- one folder you name

Fetch

- a web page, as clean text

Databases

- read-only queries

Memory

- notes that outlive the chat

- bottom row — **official reference servers**
- Zotero and QGIS — **written by volunteers**
- scite — **a paid service** , three different things to check

Adding an MCP server in Zed

the easy way **Settings → AI → MCP Servers**

1. **Install from Extensions**
2. Zed asks for whatever it needs — a key, a path
3. Watch for the **green dot**: the server is running

Zed calls them **MCP servers** and **context servers** interchangeably. Same thing.

or by hand **settings.json**

```
{
  "context_servers": {
    "zotero": {
      "command": "zotero-mcp",
      "args": [],
      "env": {}
    },
    "scite": {
      "url": "https://api.scite.ai/mcp",
      "headers": {
        "Authorization": "Bearer …"
      }
    }
  }
}
```

Local — a command Zed starts for you. That is Zotero and QGIS.
Remote — a URL you sign in to. That is scite.

BEFORE YOU INSTALL ONE

A server runs on your machine, as you, and can reach whatever you can reach.

Which is the same judgement you already make about a QGIS plugin or an R package. **Who wrote it. What does it say it can do. Does it want a key to something that matters.**

Prefer the **read-only** version when there is one. And keep the agent asking before it acts — then you see every call before it happens.

BLOCK 3

Subagents

Why an agent would start a second agent — and what it costs.

0 1 2 3 4 5 6 7 8

0:37 — 3 minutes

A subagent has its own context window



forty-one files were read. Your conversation gained one sentence.

And it hands back only the answer.

The searching happened somewhere else. You got the sentence.

When to use a subagent

Send one

- “where is the projection set in this project?”
- “which files mention the 2019 season?”
- “read these ten reports, what do they disagree about?”

`much reading, one small answer`

Do not

- “fix the bug on line 64”
- “rename this column”
- “explain what this function does”

`you already know where to look`

- it runs a whole second conversation — **slower and dearer**
- it cannot ask you anything; it knows only what it was sent with
- a vague brief returns a **confident, vague answer**
- and you cannot see what it looked at

BLOCK 4

Instructions: AGENTS.md

A new conversation knows nothing about you, your project or yesterday.

0 1 2 3 **4** 5 6 7 8

0:40 — 13 minutes

AGENTS.md is read at the start of every conversation

AGENTS.md
one file, in the project folder



And it is read at the start of every conversation. Yours, and everybody else’s.

This is the highest-value thing you will do all session.

What belongs in AGENTS.md

Put this in

```
## How we work
Ask before changing anything in data/.
Never commit. I do that.

## This project
Coordinates are EPSG:5514, not WGS84.
Finds are one row per object, not per bag.
scripts/ is throwaway. src/ is not.
```

things you have already had to say twice

Keep this out

```
Be helpful and concise.
Write clean, readable code.
Follow best practices.
Think step by step.

[three more pages of this]
```

true of everything, so it steers nothing

A long file is a weak file. Every line competes with every other line.

Where Zed looks for instructions

in the project **first match wins**

```
.rules
.cursorrules
.windsurfrules
.clinerules
.github/copilot-instructions.md
AGENT.md
AGENTS.md   ← use this one
CLAUDE.md
GEMINI.md
```

The long list is **compatibility**, not choice — so a project set up for another editor still works. Pick **AGENTS.md**: several tools now read it, so one file steers whichever agent your colleague uses.

and for you **personal**

```
# how you like to work, everywhere
~/.config/zed/AGENTS.md

# example
Explain before changing anything.
I am not a programmer — say what a
command will do before running it.
```

When they disagree, the project wins

which is right — the project is shared, and your preferences are not

Personal file: **how you work**.
Project file: **facts about the work**.

TRY IT NOW · 6 MINUTES

Finish the AGENTS.md

1. Open **AGENTS.md**. It is already half written — read it first.
2. Fill in the five **TODO** lines at the bottom. **Facts, not adjectives.**
3. At least one must be something the agent got **wrong** in the last half hour.
4. Start a **fresh conversation** and ask the question below. It should already know.

```
> What do you need to be careful about in this project?
```

BLOCK 5

Skills, and what you type

The two remaining levers — and how to tell which one a sentence belongs in.

0 1 2 3 4 5 6 7 8

0:54 — 18 minutes

What a skill is

a folder `.agents/skills/`

```
.agents/skills/
  visualize/
    SKILL.md ← the whole thing
  classify-image/
    SKILL.md
```

- in the **project**, so it travels with the work
- or in your **home folder**, for ones you want everywhere
- it is a Markdown file. There is nothing to learn

one file `SKILL.md`

```
---
name: visualize
description: Build a self-contained
  HTML page in artifacts/ that shows
  something visually. Use whenever the
  user asks to see, show, plot or draw
  anything.
---

# Make a visual artefact

One HTML file, in artifacts/, that opens
in a browser and needs nothing else.

## Rules
- One file. Everything inline.
- No internet. Conference wifi.
- Say the path when you are done.
```

- **name** — what you call it
- **description** — **when to use it**. The only part the agent always sees
- everything below — your instructions, in your words

When to write a skill

The third time you explain the same procedure, stop and write it down.

The order of the steps	The traps	When to stop and ask
what has to happen before what	what went wrong last time, and why	the cases nobody should guess at

The last one is what separates a skill from a note. A procedure that never says **“stop here and ask a human”** will confidently do the wrong thing at the first unusual case.

Skills are loaded only when needed

what the agent always sees

import-finds
take a new season's CSV into the database

make-figure
a publication-ready plot, our house style

check-coords
verify a projection and reproject if needed

season-report
build the annual report skeleton

two lines each — a name and what it is for. That is all it costs.

```
# make-figure

1. read the house palette from docs/style.md
2. never use a red/green pair — one in twelve
   readers cannot separate them
3. axis units always in the label, never the caption
4. export at 300 dpi, and also as .svg
5. put the script in figures/, not scripts/

# if the data has more than 8 categories
stop and ask. Do not invent more colours.
```

Instructions cost you on every turn. A skill costs you once.

Which is the whole reason to split them up.

A skill can call a model

a script `scripts/clip_classify.py`

```
# the same model as this morning,
# the same pinned weights
python scripts/clip_classify.py \
  --image brooch.jpg \
  --labels "a bronze brooch,
            a ceramic sherd,
            a stone tool"

  a bronze brooch   71.2%  ████████████
  a ceramic sherd   21.4%  ████
  a stone tool       7.4%  ▌
```

and a skill `.agents/skills/classify-image/`

- **when** to run it — "what is this object?", tagging a folder of photographs
- **at least three labels** , because two make it a coin toss
- the percentages are over **your** labels and nothing else
- there is **no "none of the above"** — add one and see what it takes
- rewording a label changes the answer — try two
- report the **margin** , not only the winner

The command is the easy half. The **judgement written next to it** is what makes it a skill.

No MCP, no plugin, no key. **Anything you can run from a terminal, an agent can use.**

TRY IT NOW · 5 MINUTES

Use a skill, then write one

1. Ask for the chart below. **Do not say the word “visualize”.**
2. It will probably **stop and ask you something** before drawing anything. Read why.
3. Answer it. Then open what appeared in **artifacts/**.
4. Read **.agents/skills/visualize/SKILL.md**. Find the part that made it stop.
5. Write your own skill. Five lines, and **one line saying when to stop and ask**.

```
> Show me how many finds there are per year in data/finds.csv.
```

Instructions and Skills, side by side

Instructions

always on

- read at the start of **every** conversation
- for **facts and constraints** — what the data is, what not to touch
- you never ask for them; they are simply there
- keep it short — every line is paid for every turn

```
in the project
AGENTS.md

and just for you
~/.config/zed/AGENTS.md
```

Skills

invoked by name

- read **only when the job comes up**
- for **procedures** — the order, the traps, when to stop
- the agent matches your request against the description
- can be as long as it needs to be

```
in the project
.agents/skills/<name>/SKILL.md

and just for you
~/.agents/skills/<name>/SKILL.md
```

Instructions are true **all the time**. Skills are true **sometimes**. If it would push AGENTS.md past a page, it is a skill.

Three places an instruction can live

Instructions

AGENTS.md — standing rules of this project

`always on, every single turn`

Skills

a procedure for one kind of job

`invoked by name, when needed`

what you type

this task, right now

`read once, then it is history`

paid always

paid when used

paid once

Three places. Putting something in the wrong one is the commonest mistake.

everything you write goes in exactly one of these three

A project fact typed into every message. A one-off instruction frozen into AGENTS.md forever.

Four things a good request contains

1. **What “finished” looks like** — the outcome, not the fix
2. **What you already tried** — and what happened
3. **Where to look** — so it does not read the whole project to find out
4. **What not to touch** — name the folder

Describe the outcome. The moment you describe the fix, you have stopped it thinking.

The same request, badly and well

What people type

```
the map is broken, fix it
```

It guesses what “broken” means, reads whatever it feels like, and changes something plausible.

What to type instead

```
map.png comes out empty. It should show
one dot per find.

I checked the CSV — the rows are there.
I think it is the projection.

The plotting is in scripts/plot_finds.py.
Do not touch anything in data/.
```

Outcome, what you know, where to look, what is off limits.

Four lines longer. It saves four tool calls, two wrong turns, and about a fifth of the conversation you were going to need later.

BLOCK 6

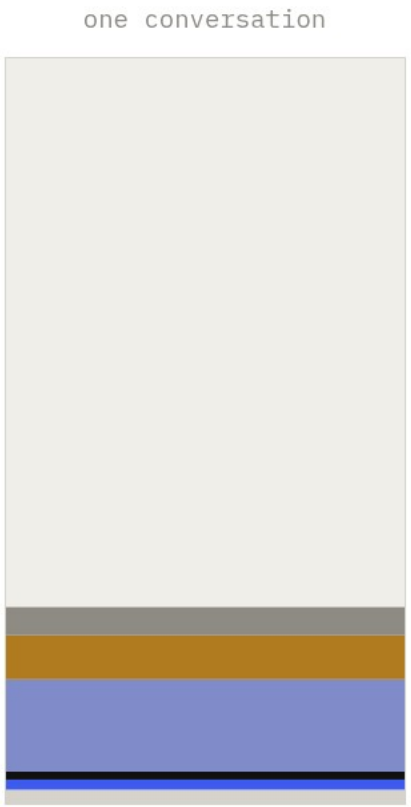
Context, compaction and lost threads

And what you never get back.

0 1 2 3 4 5 **6** 7 8

1:12 — 6 minutes

What fills the context window



the file you read at 10:04

the three approaches you rejected

why you rejected them

the exact error message

the thing you said “remember this”

gone, and it will not tell you

26%

And nothing tells you which parts went.

The agent carries on with complete confidence. This is the single most common way a long session quietly goes wrong.

Four habits that save context

Ask for a range, not a file

“look at lines 40–80” instead of “read the script”

Start again more often

a new conversation with a good AGENTS.md beats a tired one

Send someone else to search

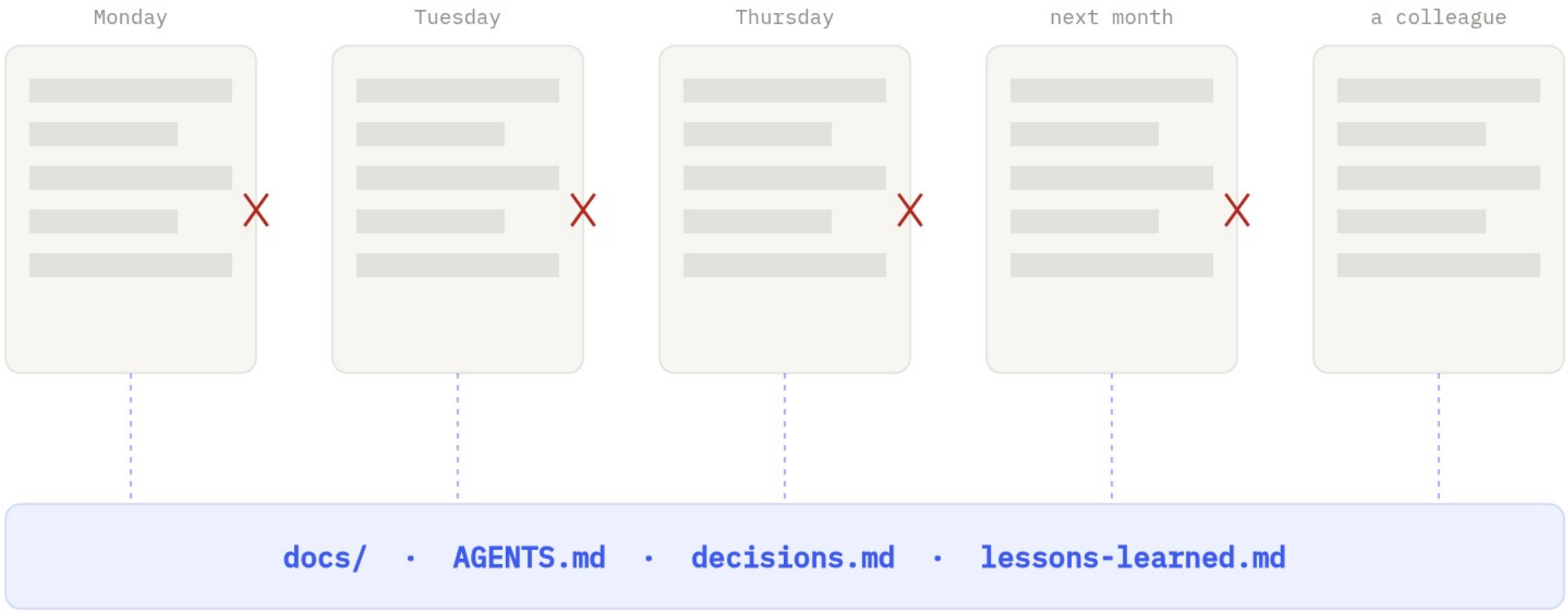
the reading lands in their window, not yours

Write conclusions to a file immediately

while the conversation still has them

A conversation is weather. A file is climate.

What survives when a conversation ends



Only what you wrote to a file crosses the gap.

And it is the same gap between you and a colleague, and between you and yourself in a year.

BLOCK 7

Decision log and lessons learned

The decision log, and the record of mistakes already made.

0 1 2 3 4 5 6 7 8

1:18 — 5 minutes

Two files, two different questions

decisions.md

- answers **why is it like this**
- one entry per real choice between real alternatives
- doing the job properly is not a decision
- append only

`read before proposing a change`

lessons-learned.md

- answers **what already went wrong**
- one entry per mistake that cost you time
- and what changed so it cannot recur
- append only

`read before repeating history`

Both are **append-only**. Nothing is ever rewritten — a decision that is overturned keeps its entry, gets its status changed, and points at the one that replaced it.

What an entry looks like

from decisions.md

```
## D-007 — The system orders, the person decides

Status: accepted

Context. A thousand clustering configurations,
none reproducing the specialist's classification.

Decision. The machine ranks candidates. A person
makes every classification, and it is recorded who.

Consequences. No automatic classification, now or
later. That question is closed.
```

from lessons-learned.md

```
## L-003 — One ignore rule hid the whole frontend

What happened. A single line in .gitignore silently
excluded 175 files, including the database schema.

How it surfaced. A commit claimed there was
nothing to commit, for a file edited minutes earlier.

What changed. Ignore rules are scoped to their
own folder, and checked after every edit.
```

The "why" line is the one that matters. In six months the *what* is obvious from the files. The *why* is gone.

WHY THIS IS NOT BUREAUCRACY

AGENTS.md points at both files. So every new conversation starts by reading them.

The agent stops suggesting the thing you ruled out in March. It stops making the mistake that cost you a day in April.

And it is the **same document** a reviewer, a supervisor and your successor need. You write it once, for three readers, one of which is a machine.

BLOCK 8

Spec-driven work with OpenSpec

The part that makes this usable for research rather than for hacking.

0 1 2 3 4 5 6 7 8

1:23 — 7 minutes

What goes wrong without a spec

Without a spec

- you describe roughly what you want
- it builds roughly that
- you describe the difference
- **repeat until tired**

`nothing to check against, nothing left afterwards`

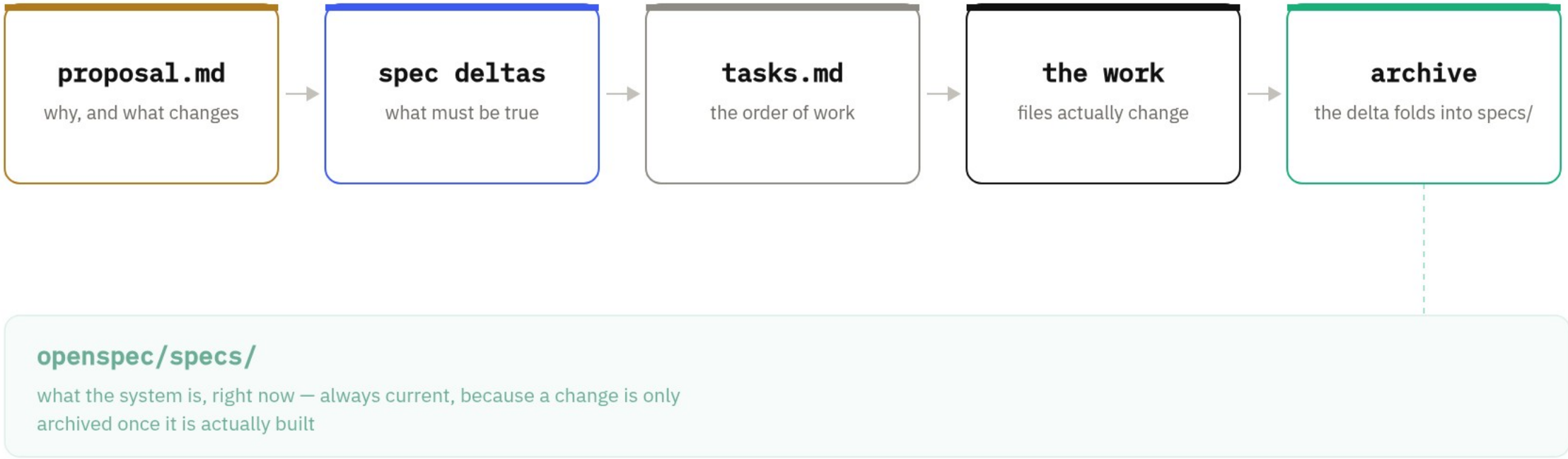
With one

- you agree **what must be true when it is done**
- it builds against that
- you check against that
- **the agreement stays** as the description

`a record that outlives the conversation`

You already do this. You call it a research design.

One change: proposal, spec, tasks, archive



And when it is done, the change is **archived**.

The requirements fold into the permanent description of the system. **The record is a by-product of doing the work.**

OpenSpec on disk, and the commands

```
openspec/
  specs/ — what is true now
    find-import/spec.md
    coordinate-handling/spec.md
  changes/ — what is being proposed
    add-season-2026/
      proposal.md
      tasks.md
      specs/find-import/spec.md
    archive/ — what was done
```

```
# once, in the project
openspec init

# ask the agent for a proposal
"propose a change for importing
 the 2026 season"

# check it is well formed
openspec validate add-season-2026

# see what is outstanding
openspec list

# when the work is finished
openspec archive add-season-2026
```

Folders of Markdown. Readable by a person with no tools at all — which is the point.

Instruction and requirement

An instruction

```
Write an import script for the
2026 season and make sure the
coordinates are handled properly.
```

only the author can tell whether it was done

A requirement

```
### Requirement: coordinates

The importer SHALL reject any row
whose coordinates fall outside the
national grid, and SHALL record the
rejected rows in a file.
```

anybody can check it — including next year

You have a name for this already.
It is an operational definition.

Five things to take home

1. **Write AGENTS.md on day one.**
2. **Make it ask before it acts**, until you can predict what it will do.
3. **When you reach a conclusion, write it to a file before the conversation ends.**
4. **The third time you explain something, make it a skill.**
5. **For anything longer than an afternoon, agree what must be true before starting.**

The agent is not the thing you are building.

The record you leave behind is.

THANK YOU

Questions